

Mindfully Yours — Implementation Strategies & Pipeline

20 — Implementation Strategies & Pipeline Flow

Purpose: [19_DATA_USAGE_MAP.md](#) answered which file goes in which bucket, and why. This document answers the next question: **for each bucket, what is the actual engineering technique that makes it work, step by step, and where exactly in the live pipeline does it fire?** Read this before writing `app/pipeline/` or `app/safety/` code — it is the bridge between "we have this data" and "here is the function that uses it."

1. The three techniques, at a glance

Bucket	Technique (the real engineering name)	One-line mechanism
1 — Teach the AI how to talk	Static few-shot prompt injection	A fixed block of text (persona + examples + the Clinical Markers table) is assembled once and inserted into every prompt — no model, no search, no runtime cost beyond token count
2 — Search and match or recommend	Embedding-based semantic retrieval (RAG)	Text is converted to a vector once at ingestion (content) and once per turn (the user's message); nearest-neighbor search over a vector index finds the closest match; a threshold decides whether to trust it
3 — Become code logic	Deterministic rule evaluation over explicit state	Plain <code>if / else</code> and ordered table lookups against numbers already sitting in Postgres/Redis — no model, no embedding, no ambiguity

Everything the AI does on a single turn is one of these three techniques, composed in a fixed order — never all three "just running RAG over everything," which is the mistake this whole 3-bucket system exists to prevent (see [04_CONVERSATION_PIPELINE.md](#) §3).

2. Bucket 1's technique in full: Static Few-Shot Prompt Injection

What it actually is

A block of text — persona description, the Clinical Markers table, and a curated handful of example conversations — assembled **once** (at app startup or per-session, not per-turn) and placed at a fixed

position in every prompt sent to the LLM. The LLM never "looks anything up" here; it reads the block like a person reading a character brief before improvising a scene.

How it's prepared (offline, once)

1. **Curate, don't dump.** From the ~30+ conversation-sample files now in Bucket 1, a human (or a one-time LLM-assisted pass, reviewed by a human) selects a small number (5–15) of the clearest, most tonally-representative examples — including at least one real crisis-handling example now that we have them (previously zero existed, see [18_NEW_CONVO_DATA_CATALOG.md](#)).
2. **Clean, don't paraphrase.** Strip drafting artifacts (leftover brackets like `[technique]`, typos, internal notes like "(could be followed up with some options)") without changing the clinical wording itself.
3. **Freeze it.** The result becomes a versioned row in `prompt_config` (see [11_DATA_MODEL_AND_STORAGE.md](#) §2) — a governed asset, not something any code path can silently edit. This is why `app/db/models/kb.py`'s `PromptConfig` model has no exposed write API (see [11_DATA_MODEL_AND_STORAGE.md](#)).

How it's consumed (runtime, every turn, but at near-zero cost)

`app/pipeline/prompt_builder.py`'s `build_prompt()` concatenates, in fixed order: persona string → Clinical Markers table (verbatim, from `app/safety/clinical_markers.py`) → curated few-shot examples → running conversation summary → trimmed recent turns → the Bucket 2/3 output for this specific turn. This function is a **pure function** — same inputs always produce the same prompt text, which is exactly why it's trivial to unit-test without a live LLM connection.

Why this technique, not retrieval

Style doesn't vary by topic — the way the AI talks about grief should be the same warmth as the way it talks about career stress. Searching for "the right tone example" per turn would be solving a problem that doesn't exist; a fixed, well-chosen set does the job for every topic at once, at zero marginal latency cost per turn (see [09_LATENCY_AND_PERFORMANCE.md](#) §7 — "keep the persona prompt lean, a bloated static prompt adds fixed latency to every call").

3. Bucket 2's technique in full: Embedding-Based Semantic Retrieval

This is the technique everyone means when they say "RAG" — but it does **two different jobs** in this system, using the exact same mechanism with different inputs and different thresholds.

The shared mechanism (identical machinery for both jobs)

1. **Chunking** — for every Bucket 2 file, one row/entry = one chunk. No further splitting needed because the client's own spreadsheets and question banks are already atomic units (see [07_INGESTION_PIPELINE.md](#) §4).
2. **Embedding** — each chunk's text is converted into a fixed-length vector (a list of numbers capturing meaning, not exact words) using the Sarvam embedding model, batched at ingestion time (see [07_INGESTION_PIPELINE.md](#) §2 step 4). This is why Sarvam was picked

even for English-only Phase 1 — switching later means re-embedding everything (see [03_TECH_STACK.md](#)).

3. **Indexing** — vectors are stored in Postgres via the `pgvector` extension, with a real approximate-nearest-neighbor index (`ivfflat`), not a linear scan (see `app/db/models/kb.py`'s `CREATE INDEX ... USING ivfflat` statements).
4. **Query-time embedding** — the user's live message is embedded with the same model, once per turn.
5. **Similarity search** — `cosine_distance()` finds the closest stored vectors. Smaller distance = more similar; `distance = 1 - cosine_similarity`.
6. **Thresholding** — a match is only trusted if `distance ≤ (1 - confidence_threshold)`. Below the threshold, the system treats it as "no match," never as a best-effort guess (per [CLAUDE.md](#) §6's core rule).

Job 2a — Matching (figure out what's going on; nothing shown to the user)

Parameter	Value	Why
What's embedded	<code>phrasings</code> (signals), <code>how_described</code> (symptoms), <code>trigger_phrases</code> (red flags), emotion phrase banks, scenario names, Layer I-IV question banks	Real, varied human phrasing — exactly what similarity search is built for
Threshold	<code>RETRIEVAL_CONFIDENCE_THRESHOLD</code> (default 0.75) for ordinary signal/scenario matching; <code>REDFLAG_MATCH_THRESHOLD</code> (default 0.55, deliberately more permissive) for red flags	A false positive on a red flag costs a moment's extra caution; a false negative costs a missed danger signal — so the bar to "trigger" is set lower on purpose (see 08_SAFETY_INTERLOCK_AND_TRIAGE.md §4)
Pre-filtering	Metadata columns (category, tier, language) filter the candidate set before the vector search runs	Narrowing first is both faster and more accurate than searching everything untagged (07_INGESTION_PIPELINE.md §3)
Output	A code (SIG-xxx, RF-xxx, an emotion label, a scenario name) — never text shown to the user	Feeds Bucket 3's rule engine, not the LLM's phrasing directly

Concrete walkthrough: user types "I don't feel like being here anymore." → embedded → compared against all 31 red-flag embeddings → closest match is RF-018 (passive suicidal ideation) at distance 0.31 → $0.31 \leq (1 - 0.55) = 0.45$ → **match confirmed** → hand off to the exact-fetch mechanism (§5 below), not to Bucket 2b, not to the LLM.

Job 2b — Recommending (real content, handed back to the user)

Parameter	Value	Why
What's embedded	Only <code>content_type IN ('psycho_education', 'self_care_tool')</code> chunks — the DBT workbook, brain-dump worksheet, <code>SELF-HELP CONVERSATIONS.docx</code> toolkit sections	The one place a "which of many things best fits" fuzzy question is genuinely being asked
When it runs	Only at Tier 1, only after Bucket 3 has already decided the tier — never runs "just in case"	Retrieval quality over quantity (09_LATENCY_AND_PERFORMANCE.md §6)
Threshold	Same <code>RETRIEVAL_CONFIDENCE_THRESHOLD</code> — an empty result below threshold routes to a safe generic fallback, never a stretched match	Never treat a low-confidence match as grounded fact

Parameter	Value	Why
Output	Up to <code>top_k=3</code> chunks, handed to the LLM as grounding material	The LLM phrases the recommendation; it does not invent which exercise to suggest

Concrete walkthrough: a Tier 1 conversation about work burnout → the conversation topic is embedded → searched against `kb_content_chunks` → top match is the DBT "PLEASE" skill (physical self-care) at distance 0.18 → within threshold → its text is handed to the LLM as grounding material → LLM phrases a warm recommendation using that content, not inventing its own.

4. Bucket 3's technique in full: Deterministic Rule Evaluation Over Explicit State

What it actually is

No model, no embedding, no fuzziness. Plain code reading rows out of Postgres (the rules) and numbers out of Redis (the conversation's current state), evaluated in a fixed order.

The three concrete mechanisms

1. **Priority-ordered table lookup** (`Kb_Phase_I_C11.csv` → `app/safety/routing.py`): rules are loaded sorted by the `priority` column (1 → 2 → 3), **never** by `rule_id` — confirmed to genuinely diverge in the real data (`05_KNOWLEDGE_BASE_DEEP_DIVE.md` §4). The first rule whose condition matches the accumulated signal set wins; evaluation stops there.
2. **Threshold checks against accumulated state** (`Kb_Phase_I_9.csv` → the sufficiency check): the set of signals observed so far this conversation (held in Redis, see `app/pipeline/state.py`) is compared against each candidate pattern's `minimum_signals` / `must_have_signals` . This is why the system "remembers" a conversation instead of judging each message alone — the state accumulates, the check re-runs every turn against the growing set.
3. **Conversation-shape checks, not text checks** (`Kb_Phase_I_17.csv` → `app/safety/fallback.py`): `trigger_condition` here means things like "15+ turns, short answers, low confidence" — read directly off Redis-tracked counters (`turn_count` , `last_response_lengths` , `confidence_score`). No text is inspected at all for this bucket's purpose.

The Pathway document becomes the control flow itself

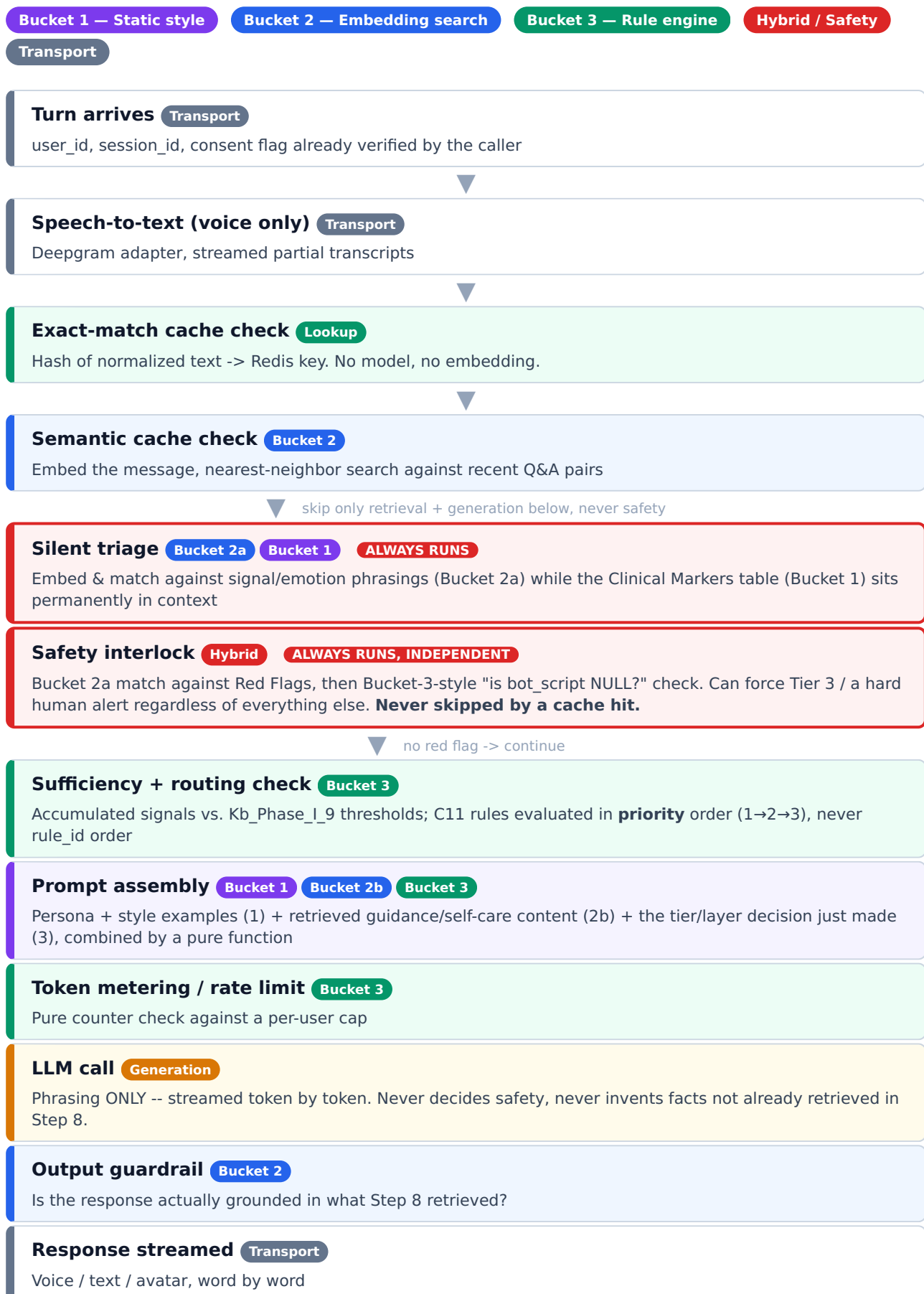
`Pathway For AI In Different Situations - Tanisha.docx` 's 15-step flow is not reference material to read at runtime — it **is** `app/pipeline/orchestrator.py` 's structure: identify concern → categorize → safety check → assess emotion/severity → choose intervention → close. Each of those 15 steps calls into either a Bucket 2 lookup (to figure out what) or a Bucket 3 rule (to decide what to do about it) — the Pathway document is the wiring diagram for how they connect, per turn.

Concrete walkthrough: signals observed this conversation = `{SIG-002}` → sufficiency check against `Kb_Phase_I_9` 's "Depression with suicidal ideation" pattern → threshold met → `evaluate_routing_rules()` scans `C11` rules in priority order → `RULE-001` (priority 1, "SIG002-003 → Tier 3") matches first → **routing decision: Tier 3**, computed in microseconds, zero model calls.

5. The hybrid case: Exact-Fetch (Bucket 2's search + Bucket 3's "no generation allowed")

The single most safety-critical mechanism in the system doesn't fit cleanly into one bucket, on purpose:

Figure — The Full Runtime Pipeline, Technique by Technique



Audit log + session close

Bucket 3

ASYNC, NON-BLOCKING

Full turn record written to Postgres; conversation state (Redis) updated for next turn -- fired after the response, never delaying it

The one rule that must never break: Steps 6-7 run on every turn, even when Step 3 or 4 already produced a cached answer. Caching (3-4) only ever skips retrieval and generation (5, 8, 10) -- never safety (6-7). See [01_PROJECT_OVERVIEW.md](#) principle #2.

The one ordering rule that must never break: Steps 6-7 (Bucket 2a matching + the Bucket 3/2 hybrid interlock) run regardless of whether Steps 3-4 (cache) already produced an answer. Caching only ever skips Steps 5/8/10 (retrieval and generation) — never 6-7 (safety). This is not a performance optimization choice, it's the core safety guarantee the whole system is built around (see [01_PROJECT_OVERVIEW.md](#) principle #2).

7. Why three techniques, not one "RAG over everything"

If everything were RAG (search-based)...	What actually happens with 3 techniques
Style examples would need to be "found" per turn — slower, and inconsistent tone across topics	Bucket 1 is loaded once, free, consistent everywhere
Precise rules (thresholds, priority order) would become fuzzy similarity matches — a routing decision could come back "close enough" instead of exactly right	Bucket 3 is exact, deterministic, and auditable — the same input always produces the same tier
Every lookup would cost an embedding call + vector search, even for things that are really just "is this number bigger than that number"	Bucket 3 checks run in microseconds, no model involved
The vector index would be bloated with content nobody ever searches (rule tables, thresholds), slowing down the searches that matter	Bucket 2's index stays lean and fast, holding only genuinely searchable content

This is the concrete payoff of the whole bucket system: **each technique is used exactly where it's the right tool, and nowhere else.**

Companion to [19_DATA_USAGE_MAP.md](#) (which file → which bucket) and [04_CONVERSATION_PIPELINE.md](#) (the full step-by-step pipeline contract). See the PDF version of this document for the visual pipeline diagram.